

Auto-encodeurs variationnels

Contenu

- Inférence variationnelle
- Auto-encodeur variationnel
- VQ-VAE

Cette page couvre deux séances de cours: [[Diapositives du cours 2](#)] et [[Diapositives du cours 3](#)]

Comme nous l'avons vu en introduction, les auto-encodeurs forment une classe de réseaux de neurones dont l'espace intermédiaire peut se prêter à la génération de données. Néanmoins, il y a une difficulté majeure : en pratique, on ignore quelle est la forme de l'espace latent, c'est-à-dire la densité $p(z)$ des codes intermédiaires. Comme la seule contrainte sur l'optimisation est l'erreur de reconstruction, cette densité peut être très irrégulière. Par exemple, les points $x_i \in \mathcal{D}$ du jeu de données peuvent chacun être projetés sur une densité piquée (un Dirac) sur z_i dans $\mathcal{Z}$ et chaque pic se trouver très éloigné des autres. Dès que l'on s'écarte des points z_i , les points décodés n'ont plus vraiment de sens. On cherche donc une *régularisation* qui permettrait de rendre l'espace latent plus continu.

Néanmoins, même si l'espace latent est continu, on ne sait a priori pas comment échantillonner dans $\mathcal{Z}$. $p(z)$ peut avoir n'importe quelle forme. Certes, il est possible de réaliser une estimation de densité à partir des points latents obtenus par encodage du jeu de données, mais cela n'est pas très efficace, surtout si l'espace latent est de grande dimension. En réalité, nous aimerions pouvoir indiquer un a priori sur la densité $p(z)$ voulue. En particulier, on aimerait pouvoir exiger que la densité p possède une certaine forme, c'est-à-dire que la fonction p appartienne à une certaine famille de fonctions. Si l'on connaît cette fonction, alors échantillonner devient plus simple.

Inférence variationnelle

Considérons un jeu de données $\mathcal{D} = \{x_1, \dots, x_n\}$ de n observations. On les supposera *i.i.d.* (indépendantes et identiquement distribuées), correspondant aux réalisations d'une variable aléatoire $x \in \mathcal{X}$. On note $p_{\mathcal{D}}$ le processus génératif qui a permis d'obtenir les observations.

Notre objectif est de construire un modèle statistique qui approxime le modèle génératif $p_{\mathcal{D}}$. Nous introduisons une variable latente $z \in \mathcal{Z}$ qui suit une distribution *a priori* $p_{\theta^*}(z)$, paramétrée par le vecteur θ^* . x_i est obtenu par la probabilité conditionnelle $p_{\theta^*}(x_i|z_i)$. L'idée est de pouvoir par la suite contrôler l'échantillonnage de $x_i \sim p$ en faisant varier le vecteur z sous-jacent.

En pratique, on ne connaît ni θ^* , ni les variables latentes $\{z_1, \dots, z_n\}$. Nous allons chercher les paramètres $\theta^* \in \Theta$ du modèle qui maximisent la vraisemblance sur $\mathcal{D}$:

$$\hat{\theta} = \arg \max_{\theta} \mathbb{E}_{p_{\mathcal{D}}}[\log p_{\theta}(x)]$$

Dans le cas général, $p_{\theta}(x) = \int p_{\theta}(x|z)p_{\theta}(z)dz$. Cette intégrale est difficile à calculer et n'a généralement pas d'expression analytique, surtout lorsque x est une variable en grande dimension. Nous ne connaissons pas non plus le postérieur $p_{\theta}(z|x)$.

Nous allons approcher le postérieur $p_{\theta}(z|x)$ par une distribution $q_{\phi}(z|x)$ paramétrée par ϕ . Si l'approximation est bonne, ce postérieur nous donnera accès à des valeurs de z étant *vraisemblablement* à l'origine de x .

Pour choisir $q_{\phi}(z|x)$, on veut s'approcher au maximum du véritable postérieur $p_{\theta}(z|x)$. On cherche par exemple à trouver les paramètres de q qui minimisent l'écart entre les deux distributions. Le choix le plus courant pour mesurer cet écart est de calculer la divergence de Kullback-Leibler entre les deux probabilités. Par conséquent, les paramètres $\hat{\phi}$ que nous recherchons sont les solutions du problème d'optimisation ci-dessous :

$$\hat{\phi} = \arg \min_{\phi} \text{KL}(q_{\phi}(z|x)||p_{\theta}(z|x))$$

La définition de la divergence de Kullback-Leibler donne :

$$\begin{aligned} \text{KL}(q(x)||p(x)) &= \int \log \frac{q(x)}{p(x)} q(x) dx \\ &= \mathbb{E}_{q(x)}[\log q(x) - \log p(x)] \end{aligned}$$

Donc, en remplaçant par les distributions qui nous concernent :

$$\begin{aligned}
\text{KL}(q_\phi(z|x)||p_\theta(z|x)) &= \mathbb{E}_{q_\phi(z|x)} [\log q_\phi(z|x) - \log p_\theta(z|x)] \\
&= \mathbb{E}_{q_\phi(z|x)} \left[\log q_\phi(z|x) - \log \frac{p_\theta(x|z)p_\theta(z)}{p_\theta(x)} \right] \quad (\text{formule de Bayes}) \\
&= \mathbb{E}_{q_\phi(z|x)} [\log q_\phi(z|x) - \log p_\theta(z) - \log p_\theta(x|z) + \log p_\theta(x)] \\
&= \underbrace{\mathbb{E}_{q_\phi(z|x)} [\log q_\phi(z|x) - \log p_\theta(z)]}_{\text{KL}(q_\phi(z|x)||p_\theta(z))} - \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] + \log p_\theta(x) \\
&= \text{KL}(q_\phi(z|x)||p_\theta(z)) - \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] + \log p_\theta(x)
\end{aligned}$$

Malheureusement dans cette formulation, nous retrouvons $p_\theta(x)$ que l'on ne connaît pas et qui n'est pas facile à calculer dans le cas général ! Toutefois, $p_\theta(x)$ ne dépend pas de ϕ : ce terme n'intervient donc pas dans notre problème de minimisation.

Cette remarque nous incite à introduire une nouvelle fonction objectif, notée

$$\text{ELBO}(q_\phi) = -(\text{KL}(q_\phi(z|x)||p_\theta(z|x)) - \log p_\theta(x)) .$$

Minimiser la divergence de Kullback-Leibler par rapport à ϕ revient à maximiser la fonction ELBO. Dans cette formulation, l'évidence $p(x)$ disparaît entièrement :

$$\begin{aligned}
\text{ELBO}(q_\phi) &= \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x)||p_\theta(z)) + \log p_\theta(x) - \log p_\theta(x) \\
&= \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x)||p_\theta(z))
\end{aligned} \tag{1}$$

Ce jeu de réécriture nous permet de remplacer le problème de minimisation de la divergence de Kullback-Leibler par un problème de maximisation de la fonction ELBO, qui ne fait intervenir que des termes connus ou contrôlables :

$$\hat{\phi} = \arg \max_{\phi} \text{ELBO}(q_\phi)$$

La maximisation de la fonction ELBO implique deux choses :

- d'une part, la maximisation de la log-vraisemblance, c'est-à-dire que les variables latentes z permettent de prédire correctement x ,
- d'autre part, la minimisation d'une pénalité correspondant à la divergence entre q et l'a priori p .

ELBO est un acronyme pour *evidence lower-bound*. En effet, la fonction ELBO est une borne inférieure de l'évidence :

$$\log p_\theta(x) = \text{ELBO}(q_\phi) + \text{KL}(q_\phi(z|x)||p_\theta(z|x))$$

et, comme la divergence KL est positive :

$$\log p_{\theta}(x) \geq \text{ELBO}(q_{\phi})$$

Cela signifie que maximiser ELBO assure que nous allons également *maximiser la vraisemblance* du modèle génératif p_{θ} .

Par souci de simplicité, nous ferons l'hypothèse que la distribution conditionnelle des variables latentes par rapport aux observations suit une loi normale. Cela fixe donc la forme de la distribution q_{ϕ} , paramétrée par un vecteur de moyennes μ et une matrice de variance-covariance Σ :

$$q_{\phi}(z|x) = \mathcal{N}(z; \mu, \Sigma)$$

Le plus souvent, on considérera que la matrice de variance-covariance est diagonale (c'est-à-dire covariances nulles) : $\Sigma = \sigma I$.

Nous ferons la même hypothèse sur la distribution a priori $p_{\theta}(z)$ des variables latentes. L'objectif est de construire un espace latent structuré facile à échantillonner. Une loi normale semble donc appropriée :

$$p_{\theta}(z) = \mathcal{N}(z; 0, I)$$

Note : d'autres distributions seraient bien entendu envisageables dans le cas où nous pouvons faire d'autres hypothèses sur l'espace latent, notamment si celui-ci est supposé discret.

Dans le cas des hypothèses gaussiennes, la différence de Kullback-Leibler a une expression analytique relativement simple à calculer :

$$\text{KL}(q_{\phi}(z|x) || p(z|x)) = \frac{1}{2} (\text{tr}(\sigma I) + \mu^t \mu - k - \log \det(\sigma I)) \quad (2)$$

Cette expression explicite va nous faciliter la vie par la suite au moment de résoudre en pratique la maximisation de la fonction ELBO.

Auto-encodeur variationnel

Jusqu'ici, nous n'avons pas vraiment évoqué comment obtenir les paramètres ϕ de l'approximation du postérieur. Reprenons le schéma de l'autoencodeur.

L'encodeur doit nous donner $p(z|x)$. Ce que nous allons faire, c'est de produire en sortie de l'encodeur non pas un code latent, mais *les paramètres de la distribution conditionnelle* $q_\phi(z|x)$. En supposant que l'espace latent $\mathcal{Z}$ soit de dimension m , alors l'encodeur aura pour sorties :

- un vecteur μ_x de longueur m , qui servira de moyenne pour la gaussienne $q_\phi(z|x)$,
- une matrice σ_x de dimensions $m \times m$, qui fera office de matrice de variance-covariance.

Ainsi, l'encodeur produit non pas un point unique mais une distribution gaussienne de points dans $\mathcal{Z}$ suivant une loi normale $\mathcal{N}(\mu_x; \sigma_x)$. On peut ensuite échantillonner selon cette loi pour obtenir des codes latents z qui seront décodés par le décodeur. Notons f la fonction représentant le décodeur. Dans notre cas, nous souhaitons minimiser l'erreur de reconstruction entre la sortie du décodeur et l'entrée de l'encodeur, c'est-à-dire que f est la solution du problème d'optimisation suivant :

$$f^* = \arg \max_f \mathbb{E}_{q_\phi(z|x)} [\log p(x|z)] = \arg \min_f \mathbb{E}_{q_\phi(z|x)} [\|x - f(z)\|]$$

Cette reconstruction correspond au premier terme de la fonction ELBO. Il faut donc ajouter le second terme correspondant à la pénalisation de la divergence KL entre l'approximation et l'a priori. Cela définit une nouvelle classe de modèle, que l'on nomme auto-encodeur *variationnel* [KW13] ou *Variational Auto-Encoder* (VAE). La fonction de coût globale de l'auto-encodeur variationnel est alors :

$$\arg \min_f \mathbb{E}_{q_\phi(z|x)} [\|x - f(z)\|] + \text{KL}(q_\phi(z|x) || p(z)) \quad (3)$$

En résumé, nous souhaitons maximiser la probabilité de reconstruire $\hat{x} \approx x$ lorsque l'on échantillonne $z \sim q_\phi(z|x)$ (terme de reconstruction) tout en contraignant la distribution latente pour une observation à ne pas trop s'éloigner de l'a priori gaussien normal (terme de régularisation).

Puisque nous avons fait l'hypothèse que q_ϕ est gaussienne, il suffit de supposer que $p(z)$ suit une loi normale pour pouvoir utiliser l'expression analytique de la divergence de Kullback-Leibler vue plus haut (2). Par conséquent, la fonction de coût globale de l'auto-encodeur variationnelle est simple à calculer. L'optimisation se fait de la manière habituelle par descente de gradient.

En pratique, la paramétrisation des distributions a posteriori $q_\phi(z|x)$ est choisie gaussienne factorisée, c'est-à-dire de covariances nulles. La matrice de variance covariances σ^2 est donc une matrice diagonale $\sigma^2 \mathbf{I}$.

Pour générer de nouvelles données, il suffit ensuite d'échantillonner des codes latents $z \sim p(z)$ suivant la loi (normale) a priori $\mathcal{N}(0, \mathbf{I})$. Ces codes latents sont ensuite passés dans le décodeur pour générer de nouvelles observations synthétiques $\hat{x}$.

Algorithme d'apprentissage

En notant E_ϕ l'encodeur de poids ϕ et D_θ l'encodeur de poids θ , la boucle d'apprentissage de l'auto-encodeur variationnel est donc la suivante :

Tant que l'optimisation n'a pas convergé ou que le nombre maximal d'itérations n'a pas atteint:

1. tirer au hasard un *batch* $x = \{x_1, x_2, \dots, x_k\}$ dans le jeu de données $\mathcal{D}$
2. **passer le *batch* dans l'encodeur**
 - calcul des paramètres de la gaussienne associée à chaque observation:
 $\mu_i, \sigma_i = E_\phi(x_i)$
3. échantillonner des codes latents pour chaque observation $z_i \sim \mathcal{N}(\mu_i, \sigma_i \mathbf{I})$
4. **passer les codes latents dans le décodeur**
 - calcul de $\hat{x}_i = D_\theta(z_i)$
5. calculer la fonction de coût ELBO $\mathcal{L}(\theta, \phi; x, \hat{x}, \mu, \sigma)$
6. calcul du gradient de ELBO et rétropropagation
7. faire une étape de descente de gradient sur θ et ϕ

Un point de blocage lors de l'implémentation de cet algorithme en pratique est la rétropropagation du gradient dans l'encodeur, c'est-à-dire le calcul de $\frac{\partial \mathcal{L}}{\partial \phi}$. Or, pour connaître ce gradient, il nous faudrait différentier le code latent z par rapport aux sorties μ, σ de l'encodeur. Mais les codes latents z sont obtenus par une échantillonnage, une opération stochastique non-différentiable. On ne peut donc a priori pas rétropropager le gradient dans l'encodeur.

Pour contourner ce problème, nous allons utiliser *l'astuce de la reparamétrisation*. Puisque c'est l'opération d'échantillonnage qui pose problème, nous devons trouver un moyen d'exprimer z en fonction de μ et σ de façon différentiable. Puisque z suit une loi gaussienne, on peut réécrire:

$$z = \mu + \sigma \odot \varepsilon \quad (4)$$

où $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$ désigne un bruit aléatoire qui suit une loi normale et $\odot$ désigne le produit vectoriel terme à terme. Ainsi, c'est désormais ε qui est échantillonné, et les dérivées partielles $\frac{\partial z}{\partial \mu}$ et $\frac{\partial z}{\partial \sigma}$ sont bien définies.

Une autre vision du VAE

La présentation théorique ci-dessus du VAE peut être intimidante car elle fait appel au cadre plus général de l'inférence variationnelle.

Reprenons l'équation (3) qui exprime le problème d'optimisation posé par l'auto-encodeur variationnel:

$$\arg \min_f \mathbb{E}_{q_\phi(z|x)} [\|x - f(z)\|] + \text{KL}(q_\phi(z|x) || p(z))$$

Cette équation se divise en deux termes. Le premier, $\|x - \hat{x}\|$, correspond à l'erreur de reconstruction entre l'entrée de l'encodeur et la sortie du décodeur. Échantillonnage de z mis à part, ce terme est strictement identique à la fonction de coût habituelle des auto-encodeurs classiques: on cherche à minimiser la perte d'information après décodage. Le second terme, $\text{KL}(q_\phi(z|x))$ peut se voir comme une régularisation qui ne dépend finalement que de μ et σ , c'est-à-dire $\mathcal{L}_{\text{reg}}(\phi; \mu, \sigma)$. L'auto-encodeur variationnel est donc une forme d'auto-encodeur *régularisé*, pour lequel on contraint la distribution $p(z)$ des codes dans l'espace latent à ne pas trop s'écarter d'un a priori gaussien.

Cette vision du VAE comme auto-encodeur régularisé amène à s'interroger sur l'équilibre entre les deux termes de la fonction objectif ELBO. Doit-on privilégier la reconstruction ou la structure de l'espace latent ? Cette réflexion amène au β -VAE (« beta-VAE ») [HMP+16], dont la formulation est identique à (3) mais inclue une pondération sur la divergence de Kullback-Leibler:

$$\arg \min_f \mathbb{E}_{q_\phi(z|x)} [\|x - f(z)\|] + \beta \cdot \text{KL}(q_\phi(z|x) || p(z)) \quad (5)$$

Dans cette formulation, il est possible de distinguer trois cas:

- lorsque $\beta = 1$, le *beta*-VAE est équivalent à l'auto-encodeur variationnel détaillé précédemment.
- lorsque $\beta > 1$, la fonction objectif accorde plus d'importance à la divergence KL, c'est-à-dire au respect de l'a priori gaussien. L'espace latent est mieux structuré. À l'inférence,

il est possible d'échantillonner des codes latents selon la loi normale a priori $p(z)$. En revanche, les erreurs de reconstruction sont moins pénalisées.

- lorsque $\beta < 1$, la fonction objectif privilégie la vraisemblance des données. Les reconstructions sont donc meilleures. Néanmoins, l'espace latent est moins structuré car la contrainte sur $q_\phi(z|x)$ est moins forte. Il est donc possible que les paramètres μ, σ des gaussiennes prédites pour certaines observations x s'éloignent beaucoup de la référence normale $\mathcal{N}(0, \mathbf{I})$. Échantillonner selon la loi a priori $p(z)$ est donc plus risqué, car les codes ainsi obtenus pourraient ne pas être représentatifs des codes générés lors de l'apprentissage du VAE.

Auto-encodeur variationnel conditionnel

L'auto-encodeur variationnel conditionnel (*conditional variational autoencoder* ou CVAE) [SLY15] est une extension du VAE. L'objectif est d'introduire plus de contrôle dans la génération de nouvelles données. En effet, pour l'instant, seul le code latent z permet de contrôler quelle donnée sera générée par le décodeur. Notre contrôle sur la sortie du décodeur est donc faible: pour changer la donnée, il faut changer z . Or, nous ne savons pas comment nos déplacements dans l'espace latent vont affecter les données générées. Il n'y a en effet pas nécessairement de lien entre les dimensions de l'espace latent et la sémantique des données.

Supposons néanmoins que pour chaque observation $x_i \in \mathcal{D}$, nous disposons d'une variable descriptive c_i qui représente une propriété intéressante de la donnée. Par exemple, y_i pourrait représenter la classe d'une image (est-ce un chat ? un chien ? un oiseau ?) ou la hauteur d'un son. Afin de contrôler la génération d'une observation x_i , on peut désormais s'intéresser à la distribution $p(x|z, c)$ conditionnée à c . Dans notre VAE, cela revient à optimiser la fonction objectif ELBO conditionnelle, en remplaçant les distributions de l'équation (1) par les distributions conditionnelles :

$$\begin{aligned}\text{ELBO}(q_\phi) &= -(\text{KL}(q_\phi(z|x, c) || p_\theta(z|x, c)) - \log p_\theta(x, c)) \\ &= \mathbb{E}_{q_\phi(z|x, c)}[\log p_\theta(x|z, c)] - \text{KL}(q_\phi(z|x, c) || p_\theta(z|c))\end{aligned}$$

La variable latente est désormais conditionnée à c , c'est-à-dire que nous avons dans cette formulation un espace latent (et une distribution $p(z)$) pour chaque valeur du conditionnement.

Concrètement, cette formulation présente les mêmes équivalences que la fonction objectif ELBO classique: maximiser la ELBO permet de maximiser la vraisemblance $p(x|c)$ et de

minimiser la divergence de Kullback-Leibler entre les distributions a posteriori $q_\phi(z|x, c)$ et $p_\theta(z|x, c)$.

En pratique, l'implémentation des VAE conditionnels ne nécessite que deux changements:

- ajouter le conditionnement c à l'entrée de l'encodeur, par exemple en le concaténant au vecteur des entrées x dans le cas de données vectorielles,
- ajouter le conditionnement c à l'entrée du décodeur en le concaténant au code latent z .

La nature précise de ces changements dépend bien entendu de l'architecture choisie pour les réseaux encodeur et décodeur.

VQ-VAE

Le VAE que nous avons vu jusqu'à présent émet l'hypothèse que l'espace latent peut se structurer autour d'une gaussienne centrée réduite. Les variables latentes sont donc continues et suivent une loi normale. Pourtant, ce n'est pas la seule façon de construire un auto-encodeur variationnel. Comme nous l'avons vu précédemment, même si choisir une loi a posteriori gaussienne (et une loi a priori gaussienne également) facilite les calculs, nous aurions pu décider d'autres hypothèses.

Le *Vector-Quantized Variational Auto-Encoder* (VQ-VAE) [\[vdOVK17\]](#) propose ainsi de compresser la distribution des observations dans un espace latent *discret*. Plus précisément, le modèle apprend une distribution catégorielle multivariée dans l'espace latent. Ainsi, au lieu d'avoir un code $\mathbf{z}$ contenant des variables latentes $z_1, z_2, \dots, z_d \in \mathbb{R}$, le code sera ici constitué d'entiers $i_1, i_2, \dots, i_d \in \{1, K\}$. Ces entiers pourront aller de 1 à K , et chaque entier représentera un « élément » d'un dictionnaire de code qui sera fixé lors de l'inférence. On parlera aussi d'*atomes* pour désigner ces éléments.

Cette représentation aura deux avantages. D'une part, l'espace latent ainsi modélisé pourra être bien plus complexe qu'une loi normale. En particulier, il sera plus facile de modéliser des distributions multimodales. D'autre part, manipuler des entiers facilite la manipulation numérique, notamment pour le stockage des codes.

Pour construire un VQ-VAE, nous devons d'abord introduire le dictionnaire, parfois appelé *codebook*. Nous allons remplacer l'espace latent $\mathcal{Z}$ par un dictionnaire de K codes latents de dimension d (K et d sont des hyperparamètres à choisir) :

$$\{e_1, e_2, \dots, e_K\}$$

Une observation x sera alors transformée par l'encodeur en un vecteur de n entiers entre 1 et K . En entrée du décodeur, on remplacera ce vecteur par le vecteur obtenu en concaténant les atomes du dictionnaire correspondant aux entiers codés. Par exemple :

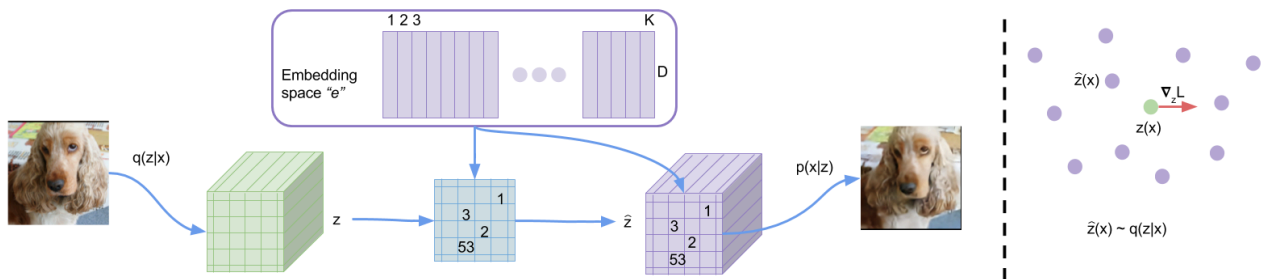
$$x \rightarrow [1, 7, 104, 12] \rightarrow [e_1, e_7, e_{104}, e_{12}]$$

En pratique, l'encodeur ne produit pas directement des entiers. En réalité, l'encodeur est inchangé par rapport au VAE classique : il produit un code z de dimension d (la même que celle des codes du dictionnaires). Toutefois, ce code sera ensuite discrétisé et donc remplacé par l'atome du dictionnaire le plus proche. Ainsi, la distribution postérieure $q_\phi(z|x)$ obtenue par l'encodeur est obtenue par :

$$q_\phi(z = k|x) = \begin{cases} 1 & \text{si } k = \arg \min_j \|z_e(x) - e_j\|_2 \\ 0 & \text{sinon} \end{cases}$$

Autrement dit, l'encodeur donne $z_e = E(x)$ qui est remplacé par le vecteur e_i qui lui est le plus proche dans l'espace $\mathcal{Z}$. L'espace latent est ainsi sous-divisé en K parties. D'une certaine façon, cela revient à définir un partitionnement de l'espace similaire à celui d'un k-means, en utilisant les atomes du dictionnaire comme centres. La « catégorie » latente est obtenue par recherche du plus proche voisin. À noter que les codes e_i (les éléments du dictionnaire) seront initialisés aléatoirement mais ensuite appris pendant l'entraînement.

En pratique, $q_\phi(z = k|x)$ est une distribution catégorielle multivariée, c'est-à-dire que l'encodeur renvoie n codes z_e et que chaque code est quantifié à l'aide de l'opération précédente. Le code latent complet est ainsi constitué de n entiers entre 1 et K .



Optimisation du VQ-VAE

Puisque nous avons modifié la loi postérieure, la fonction objectif ELBO pour le VQ-VAE change par rapport au VAE classique. Rappelons que la loi postérieure est désormais :

$$q_\phi(z = k|x) = \begin{cases} 1 & \text{si } k = \arg \min_j \|z_e(x) - e_j\|_2 \\ 0 & \text{sinon} \end{cases}$$

Cela signifie que $q_\phi(z = k|x)$ est *déterministe*. Nous allons supposer que la loi a priori sur les codes latents - c'est-à-dire l'atome le plus proche - est *uniforme* (autrement dit, qu'en moyenne, tous les éléments du dictionnaire ont autant de chance d'être choisis pour encoder une observation). Alors, dans ce cas, on peut montrer que la divergence de Kullback-Leibler de la loi postérieure par rapport à la loi a priori a une expression simple :

$$\text{KL}(q_\phi(z|x)||p_\theta(z)) = \log K$$

En réinjectant dans la fonction ELBO, la fonction objectif du VQ-VAE est alors :

$$\mathcal{L}(\theta, \phi; x) = \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]}_{\log p_\theta(x|z) \text{ car } q_\phi(z|x) \text{ est déterministe}} - \underbrace{\text{KL}(q_\phi(z|x)||p_\theta(z))}_{\log K}$$

Puisque $\log K$ est une constante, il ne reste plus qu'à optimiser la vraisemblance :

$$\mathcal{L}(\theta, \phi; x) = \log p_\theta(x|z)$$

Pour que l'apprentissage du VQ-VAE soit possible, il faut toutefois apporter un soin particulier à l'opération de quantification. En effet, le décodeur reçoit en entrée un code $e_i = z_q(x)$ qui est, par définition, l'atome qui est le plus proche de voisin de $z_e(x)$ la sortie de l'encodeur :

$$z_q(x) = \arg \min_{e_j \in \{e_1, \dots, e_K\}} \|z_e(x) - e_j\|$$

Or, l'opération $\arg \min$ est *non-dérivable*. Pourtant, il nous faut bien calculer le gradient de z_q par rapport à z_e afin d'effectuer la rétropropagation. La solution choisie est de simplement « copier » le gradient en entrée du décodeur D_θ à la sortie du décodeur E_ϕ . Autrement dit, on applique à z_e le gradient par rapport à z_q , comme si ces deux variables étaient égales. Bien qu'il s'agisse d'une approximation, cette astuce fonctionne en pratique.

Pour apprendre les atomes, nous allons utiliser le principe suivant : si une observation x passée dans l'encodeur donne un vecteur z_e , et que e est l'atome le plus proche de z_e , alors cet atome doit se déplacer vers z_e . Autrement dit, cela revient à ajouter une pénalité quadratique sur la distance entre le code et l'atome le plus proche :

$$\mathcal{L}(\theta, \phi; x) = \underbrace{\log p_{\theta}(x|z_q(x))}_{\text{vraisemblance}} + \underbrace{\|\text{sg}[z_e(x)] - e\|_2^2}_{\text{apprentissage des atomes, sg = "stop gradient"}}$$

Ici, *stop gradient* signifie que le gradient n'est pas rétropropagé dans l'encodeur via z_e (le code est traité comme une vérité terrain du point de vue de l'apprentissage des atomes).

Une difficulté qui peut apparaître dans cette formulation est que les codes de l'encodeur peuvent fluctuer et s'éloigner petit à petit des atomes du dictionnaire. Les atomes sont ensuite optimisés par le deuxième terme de la fonction objectif pour rattraper les codes, mais rien ne force l'encodeur à rester autour des atomes. Pour contrecarrer ce phénomène, il est possible d'introduire une *commitment loss*, pour forcer l'encodeur à « s'engager » sur le choix des atomes :

$$\mathcal{L}(\theta, \phi; x) = \underbrace{\log p_{\theta}(x|z_q(x))}_{\text{vraisemblance}} + \underbrace{\|\text{sg}[z_e(x)] - e\|_2^2}_{\text{apprentissage des atomes}} + \beta \underbrace{\|z_e(x) - \text{sg}[e]\|_2^2}_{\text{commitment loss}}$$

Ainsi, l'encodeur est pénalisé si les codes prédits sont trop éloignés de leur plus proche atome. Cela permet donc de rapprocher les atomes des codes, mais aussi de rapprocher les codes des atomes, afin de forcer la convergence du partitionnement de l'espace latent.

En pratique

En pratique, le VQ-VAE utilise plusieurs entiers pour une seule observation, c'est-à-dire qu'une observation x est codée par n vecteurs latents, et donc n atomes. Typiquement, une image de dimensions 224×224 sera encodée par une matrice de 32×32 entiers.

Cette représentation rend l'échantillonnage plus complexe. En effet, l'a priori uniforme indiquerait qu'il suffit d'échantillonner au hasard n entiers entre 1 et K . Malheureusement, les n entiers qui codent une observation sont rarement indépendants. À la place, la solution généralement adoptée consiste à introduire un modèle autorégressif (chaîne de Markov, PixelCNN, RNN, etc.) entraîné sur l'ensemble des séquences d'atomes qui codent les observations du jeu d'entraînement, puis d'échantillonner selon la distribution apprise par ce modèle.

Pour aller plus loin

[[KW19](#)] est un tutoriel très complet des auteurs de la formulation originale du VAE concernant les auto-encodeurs variationnels, incluant une discussion d'a priori non gaussiens.

Références

- [[HMP+16](#)] Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, and Alexander Lerchner. Beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework. In *International Conference on Learning Representations*. November 2016.
- [[KW13](#)] Diederik P. Kingma and Max Welling. Auto-Encoding Variational Bayes. In *International Conference on Learning Representations*. December 2013.
- [[KW19](#)] Diederik P. Kingma and Max Welling. An Introduction to Variational Autoencoders. *Foundations and Trends in Machine Learning*, 12(4):307–392, 2019. [arXiv:1906.02691](#), [doi:10.1561/22000000056](#).
- [[SLY15](#)] Kihyuk Sohn, Honglak Lee, and Xinchen Yan. Learning Structured Output Representation using Deep Conditional Generative Models. In *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015.
- [[vdOVK17](#)] van den Oord, Aaron, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NeurIPS'17, 6309–6318. 2017.